# Planeamento e Gestão de Projeto

Ano Letivo 2025/2026

# Tema do Projeto

**Relatório da Etapa 2**


Grupo 08

| | |
|---|---|
| Francisco Parasca | 62233 |
| Miguel Lemos | 62248 |
| Rodrigo Sequeira | 62255 |
| Gonçalo Moutinho | 62256 |
| Martim Melro | 62267 |

# Índice


Introdução 3

1. Recursos 3

   1.1. Pessoas da equipa 3

   1.2. Ferramentas de desenvolvimento 3

   1.3. *Software* para o funcionamento da aplicação *web* 3

2. Estimativas 4

   2.1. Esforço disponível 4

   2.2. Dados históricos 4

   2.3. Estimativas baseadas em linhas de código 4

   2.4. Estimativas baseadas no modelo COCOMO 4

3. Processo de desenvolvimento de *software* 5

4. Organização da equipa 5

5. Planeamento do projeto 5

   5.1. *Work packages* 5

   5.2. Calendarização 6

6. Gestão de riscos 6

   6.1. Lista de riscos 6

   6.2. Tabela de riscos 6

   6.3. Plano RMMM 6

Conclusão 6

Bibliografia 6

Apêndice 7

# Introdução

O presente relatório enquadra-se na Unidade Curricular de Planeamento e Gestão de Projeto e tem como objetivo principal apresentar o planeamento detalhado para o desenvolvimento de uma aplicação *web* destinada à gestão de uma empresa de táxis.

O problema a resolver, descrito no enunciado do projeto, consiste na necessidade de digitalizar e otimizar os processos de negócio de uma frota de táxis. Atualmente, a gestão de motoristas, turnos, viaturas (elétricas e a combustão), bem como o registo de viagens e faturação a clientes, carece de uma solução integrada. O sistema proposto visa colmatar estas falhas, permitindo uma gestão centralizada para o gestor, uma operação fluida para o motorista e uma experiência de pedido de táxi eficiente para o cliente.

As mais-valias deste projeto residem na automação de tarefas críticas, como o cálculo automático de preços com base no tempo e conforto , a gestão de restrições de turnos e a validação de dados em tempo real, garantindo a integridade da informação e o cumprimento de requisitos legais como a emissão de faturas. Tecnologicamente, a solução apoia-se numa arquitetura cliente-servidor, utilizando **React.js** para o *frontend* e **SpringBoot** para o *backend*, conforme definido na Etapa 1.

Este relatório cobre o planeamento da Etapa 2, abordando a identificação dos recursos humanos e técnicos, as estimativas de esforço baseadas em dados históricos e no modelo COCOMO II, a definição do processo de desenvolvimento, a organização da equipa e a gestão de riscos associada ao projeto.

# 1. Recursos

## 1.1. Pessoas da equipa

A nossa equipa de desenvolvimento do projeto é composta por 5 elementos. Abaixo apresenta-se a lista dos membros e o resumo das suas atividades paralelas que possam impactar a disponibilidade para este projeto:

- Francisco Parasca (62233): Inscrito a 5 unidades curriculares, sendo 3 na FCUL (entre elas PTIR - Projeto de Tecnologias de Informação e Redes) e 2 na FBAUL no âmbito do Minor em Design Multimédia. Esta situação obriga a deslocações frequentes entre os campus da Cidade Universitária e do Chiado. Para além disso, mantém uma disciplina desportiva rigorosa, praticando MMA todos os dias durante o período noturno. Devido às deslocações diurnas e à ocupação das noites, a sua disponibilidade concentra-se nos períodos livres entre aulas e predominantemente aos fins de semana.
- Miguel Lemos (62248): Inscrito a 5 unidades curriculares, sendo 3 delas na FCUL (entre elas PTIR - Projeto de Tecnologias de Informação e Redes) e 2 no ISEG no âmbito de Minor em Gestão. Esta situação obriga a deslocações frequentes entre os campus do Campo Grande e do Instituto Superior de Economia e Gestão, em Santos, reduzindo assim as janelas de tempo livre durante o dia.
- Rodrigo Sequeira (62255): Inscrito a 7 unidades curriculares, todas na FCUL (entre elas PTIR - Projeto de Tecnologias de Informação e Redes). Mantém uma atividade remunerada diária de apoio logístico familiar, ocupando cerca de 1:30h por dia, no período do fim do dia. Pratica também futebol, onde tem 2 jogos por semana. Estas

responsabilidades impõem restrições horárias fixas, que foram consideradas no cálculo da sua atividade semanal.

- Gonçalo Moutinho (62256): Tal como o Miguel, frequenta 5 unidades curriculares divididas entre a FCUL (3) e o ISEG (2), enfrentando os mesmos desafios logísticos de deslocação e calendários de avaliação mistos. Para além disso é Escuteiro (ocupação fixa aos sábados), participa num Grupo de Jovens (terças à noite) e mantém uma rotina de treino de atletismo/corrida 3 vezes por semana. Devido a esta agenda preenchida, a sua disponibilidade concentra-se fortemente nos dias úteis (exceto terças ao final do dia) e aos Domingos, exigindo um planeamento rigoroso das reuniões de equipa para não colidir com as atividades.
- Martim Melro (62267): Inscrito a 5 unidades curriculares, sendo 3 delas nucleares do curso (entre elas PTIR - Projeto de Tecnologias de Informação e Redes) e as restantes 2 no âmbito do Minor em Tecnologias de Informação Geográfica. Para além das responsabilidades académicas, mantém uma rotina desportiva intensa, integrando 2 equipas de futebol distintas (o que implica 2 treinos e 2 jogos semanais) e frequentando regularmente o ginásio. Devido a esta carga física e horária, a sua disponibilidade é reduzida durante os dias de treino e fins de semana.

**Disponibilidade da Equipa:**

Com base nas restrições acima e consultando o calendário acadêmico do ano anterior para ter em consideração datas de avaliações das unidades curriculares e pausas letivas, estimou-se a disponibilidade percentual de cada elemento do grupo.

Considerou-se as semanas 6 e 7 como as pausas de celebração da Páscoa, ocorridas entre os dias 30 de março e 10 de abril.

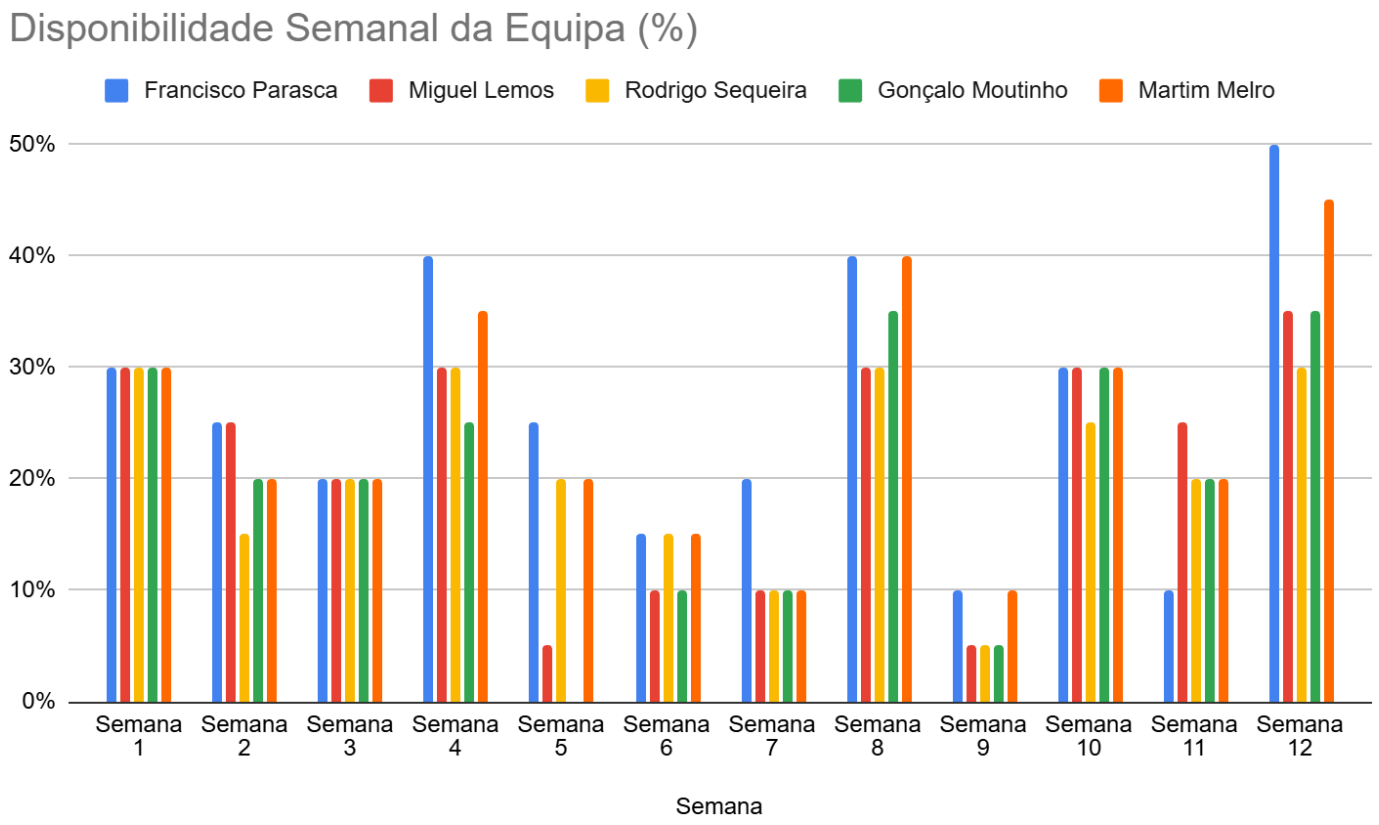
É visível a existência de três picos de disponibilidade nas Semanas 4 (32%), 8 (35%) e 12 (38%). Estes aumentos de carga horária correspondem diretamente às semanas que antecedem as Reuniões de Avaliação de PTIR. Nestes períodos, a equipa adota uma lógica de *sprint*, priorizando as entregas do projeto face a outras obrigações menos urgentes, garantindo assim o cumprimento dos requisitos funcionais e não funcionais exigidos.

Por outro lado, verificam-se quebras notórias nas semanas imediatamente a seguir às avaliações (Semanas 5 e 9). Esta redução justifica-se pelo desgaste acumulado na semana de entrega anterior, levando a equipa a reduzir temporariamente o foco neste projeto para recuperar o atraso nas restantes unidades curriculares. Adicionalmente, este período de menor disponibilidade coincide também com a sobrecarga dos Minors na FBAUL e no ISEG e com a pausa letiva da Páscoa, exigindo uma gestão de tempo mais compartimentada.

Tabela 1 - Disponibilidade semanal (percentagem do tempo total de trabalho semanal):

| Semana | Datas Aprox. | Francisco Parasca | Miguel Lemos | Rodrigo Sequeira | Gonçalo Moutinho | Martim Melro | Média Equipa |
|---|---|---|---|---|---|---|---|
| Semana 1 | 22-28 fev | 30% | 30% | 30% | 30% | 30% | 30% |
| Semana 2 | 01-07 mar | 25% | 25% | 15% | 20% | 20% | 21% |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| Semana 3 | 08-14 mar | 20% | 20% | 20% | 20% | 20% | 20% |
| Semana 4 | 15-21 mar | 40% | 30% | 30% | 25% | 35% | 32% |
| Semana 5 | 22-28 mar | 25% | 5% | 20% | 0% | 20% | 14% |
| Semana 6 | 29-04 abr | 15% | 10% | 15% | 10% | 15% | 13% |
| Semana 7 | 05-11 abr | 20% | 10% | 10% | 10% | 10% | 12% |
| Semana 8 | 12-18 abr | 40% | 30% | 30% | 35% | 40% | 35% |
| Semana 9 | 19-25 abr | 10% | 5% | 5% | 5% | 10% | 7% |
| Semana 10 | 26-02 mai | 30% | 30% | 25% | 30% | 30% | 29% |
| Semana 11 | 02-09 mai | 10% | 25% | 20% | 20% | 20% | 19% |
| Semana 12 | 10-16 mai | 50% | 35% | 30% | 35% | 45% | 38% |
| Média Semanal | ——— | 26.3% | 21.3% | 20.8% | 20% | 24.6% | 22.6% |



A Tabela 2 apresenta as competências técnicas e pessoais dos membros da equipa, classificadas de acordo com a seguinte escala:

- 1 (Iniciante): Conhecimento teórico básico, sem experiência prática.
- 2 (Júnior): Conhecimento básico com alguma experiência académica.
- 3 (Intermédio): Autónomo na maioria das tarefas, requer ajuda em problemas complexos.
- 4 (Avançado): Capacidade de resolver problemas complexos e apoiar outros.
- 5 (Especialista): Domínio completo da tecnologia ou competência.

Tabela 2: Matriz de Competências

| Tipo | Competência | Francisco Parasca | Miguel Lemos | Rodrigo Sequeira | Gonçalo Moutinho | Martim Melro |
|---|---|---|---|---|---|---|
| Técnicas | Java | 5 | 4 | 3 | 3 | 4 |
| | JavaScript | 4 | 3 | 4 | 4 | 4 |
| | Python | 5 | 5 | 4 | 4 | 2 |
| | SQL | 2 | 5 | 4 | 5 | 4 |
| | Git | 4 | 4 | 4 | 3 | 4 |
| | Docker | 4 | 3 | 2 | 2 | 3 |
| Pessoais | Liderança | 4 | 4 | 3 | 5 | 4 |
| | Comunicação | 3 | 4 | 4 | 5 | 4 |
| | Gestão de Tempo | 3 | 3 | 3 | 3 | 3 |
| | Redação Técnica | 1 | 5 | 4 | 4 | 4 |

## 1.2. Ferramentas de desenvolvimento

O hardware que será utilizado para a realização deste projeto inclui computadores portáteis, desktops e periféricos para complementar os desktops.

Editores de código: Visual Studio Code [10] e IntelliJ [1] foram utilizados anteriormente, caracterizados por uma interface intuitiva e suporte para extensões.

Documentação de linguagens: Java [2], SQL [3] e JavaScript [4].

Bibliotecas: React.js [5], Leaflet.js [6], Spring Boot [7], Spring Data [8], Spring Security [9], Stripe API [11], Firebase Authentication SDK [12], Nominatim [13] e OSRM [14].

Frameworks de testes: JUnit [15], reconhecido pela sua simplicidade e eficácia.

Geradores de documentação: Javadoc [16] e Swagger/OpenAPI [17] são altamente recomendados para documentação profissional.

Controlo de versões: Git [18], com base na experiência adquirida durante o curso e na sua simplicidade de utilização.

Ferramentas de videoconferência: Discord [19], utilizado para comunicação entre o grupo e organização do trabalho.

## 1.3. *Software* para o funcionamento da aplicação *web*

O Software a ser usado neste projeto é baseado na nossa experiência na duração do curso até à atualidade e nas capacidades dos membros membros do grupo de utilizar estes softwares.

Como servidor web escolhemos utilizar Nginx devido à sua integração fácil com React.js e é compatível com todas as necessidades que a aplicação irá precisar.

Optamos por utilizar o servidor Tomcat/Jetty embutido, que faz parte do ecossistema Spring Boot. Esta abordagem simplifica a arquitetura e o deploy da nossa aplicação, uma vez que o Spring Boot é projetado para incluir o servidor necessário para executar a API REST, o que é eficiente e padrão em aplicações Java modernas.

Como SGDB (Sistema de gestão de bases de dados) escolhemos utilizar o PostgreSQL devido à sua simplicidade de trabalho e a nossa exposição a esta SGDB no decorrer do curso.

E por fim, vamos utilizar serviços de computação em nuvem da AWS, mais em específico o EC2 para hospedar o que for necessário, RDS para a base de dados e ECS para os containers.

Houveram algumas alterações do que tinha ficado como escolha na entrega anterior.

# 2. Estimativas

## 2.1. Esforço disponível

O cálculo do esforço total que a equipa pode dedicar ao projeto foi realizado com base nas disponibilidades semanais detalhadas na Secção 1.1 e na duração prevista do projeto.

**Premissas de Cálculo:**

- Duração do Projeto: Considerou-se o intervalo de 12 semanas, com início na semana de 22 de fevereiro de 2025 e término previsto para 16 de maio de 2025;
- Base Horária: Foi considerada uma semana de trabalho padrão de 40 horas como referência (100%);
- Fórmula: O esforço individual resulta do somatório das horas disponíveis em cada uma das 12 semanas, ajustado às flutuações de disponibilidade (ex: pausas letivas e entregas intermediárias).

A Tabela 3 apresenta o esforço total estimado por elemento e o total acumulado da equipa.

Tabela 3: Esforço disponível da equipa

| Elemento da Equipa | Perfil / Restrição Principal | Média Horas / Semana | Esforço Total (Horas) |
|---|---|---|---|
| Francisco Parasca | FCUL + FBAUL + Horas de transportes | 10.5h | 126h |
| Miguel Lemos | FCUL + ISEG | 8.5h | 102h |
| Rodrigo Sequeira | FCUL + Atividade Familiar Remunerada + Atividade Desportiva | 8h | 96h |
| Gonçalo Moutinho | FCUL + ISEG + Escuteiros + Grupo de Jovens + Atividade Desportiva | 7h | 84h |
| Martim Melro | FCUL + Atividade Desportiva | 9h | 108h |
| Total | ——— | 43h | 516h |

**Análise de esforço:**

A equipa dispõe de um banco total de 516 horas para a realização de todas as etapas do projeto (desenho, implementação, testes e documentação), considerando o período efetivo de 12 semanas de desenvolvimento do projeto.

Este valor reflete a capacidade real de trabalho do grupo, já ponderando as limitações logísticas decorrentes da frequência de unidades curriculares em diferentes instituições (FCUL,

ISEG e FBAUL), o que reduz significativamente as janelas de oportunidade para trabalho presencial e síncrono durante os dias úteis.

Adicionalmente, o planeamento integra o impacto das diversas atividades extracurriculares regulares dos membros, que variam entre desporto, voluntariado e atividades profissionais parciais. Desta forma, considera-se que estas 516 horas constituem o limite de esforço sustentável para garantir a qualidade da entrega sem comprometer o rendimento académico nas restantes cadeiras do semestre.

## 2.2. Dados históricos

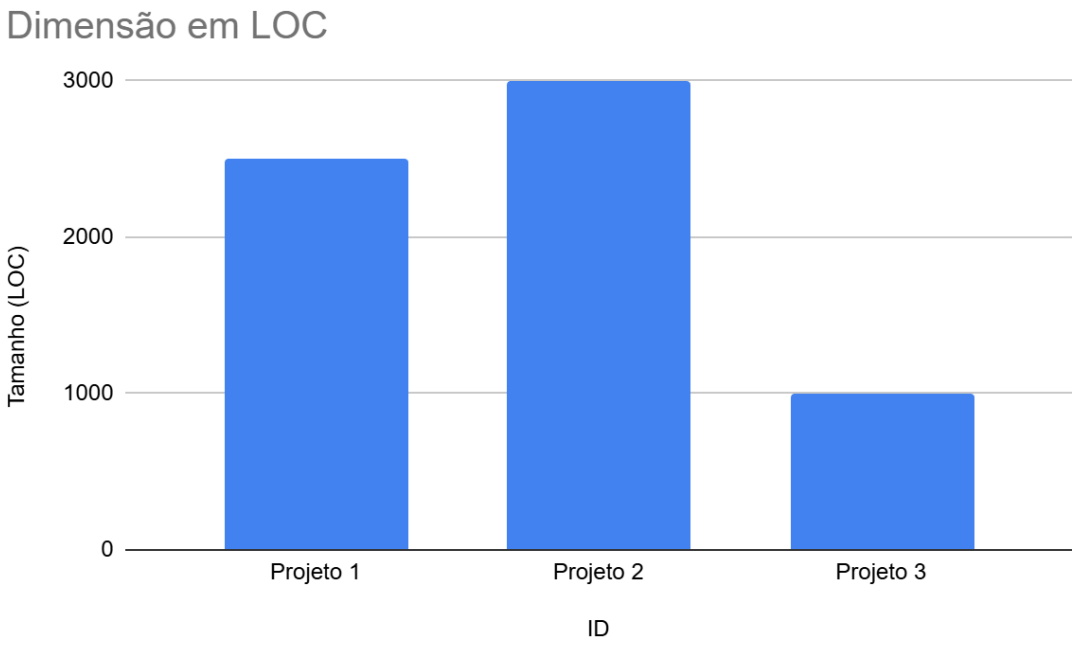
Para obter uma estimativa realista da capacidade produtiva da equipa, foram recolhidos dados métricos de projetos académicos realizados anteriormente pelos membros do grupo. Foram selecionados três projetos de referência, desenvolvidos no último ano letivo, que partilham semelhanças técnicas ou de complexidade com o projeto atual de PGP.

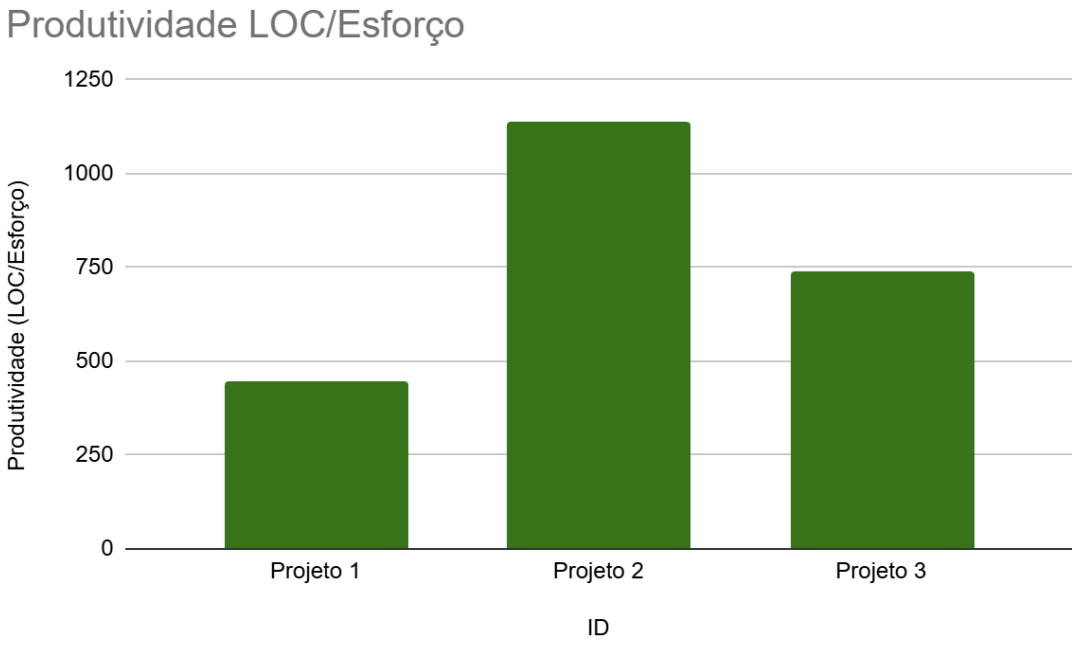
**Descrição dos Projetos Analisados:**

- Projeto A (Aplicações e Serviços na Web): Realização de uma aplicação (e serviços) Web EcoRide de partilha de viagens (ridesharing) que promove uma forma mais eficiente, económica e sustentável de viajar. Dividido em 3 etapas.
- Projeto B (Construções de Sistemas de Software): Desenvolvimento incremental de um ecossistema de software composto pelo serviço de meteorologia "WeatherWise" e pela plataforma de gestão de bicicletas partilhadas "UrbanWheels". O projeto abrangeu a implementação de lógica de negócio complexa (gestão de frota, alugueres e manutenção), exposição de API REST e desenvolvimento de interfaces Web (Thymeleaf) e Desktop (JavaFX) integradas. Dividido em 3 fases.
- Projeto C (Programação Centrada em Objetos): Desenvolvimento de uma aplicação que simula os jogos de lógica Mastermind e Bulls and Cows, aplicando conceitos de Programação Orientada a Objetos para implementar a gestão de tentativas ilimitadas e a validação algorítmica de sequências secretas geradas pelo computador.

Tabela 4: Dados Históricos de Projetos Anteriores

| ID | Unidade Curricular | Linguagem | Duração (Meses) | Nº de Elementos | Disponibilidade Média (%) | Esforço (Pessoa/ Meses) | Tamanho (LOC) | Produtividade (LOC/Esforço) |
|---|---|---|---|---|---|---|---|---|
| Projeto 1 | ASW | HTML, PHP e JavaScript | 3.5 | 4 | 40% | 5.6 | 2500 | 446.43 |
| Projeto 2 | CSS | Java e SQL | 2.5 | 3 | 35% | 2.63 | 3000 | 1140.68 |
| Projeto 3 | PCO | Java | 1.5 | 3 | 30% | 1.35 | 1000 | 740.74 |
| Médias | ——— | ——— | 2.5 | 3.3 | 35% | 3.2 | 2166.7 | 776 |
| Mínimos | ——— | ——— | 1.5 | 3 | 30% | 1.35 | 1000 | 446.4 |
| Máximos | ——— | ——— | 3.5 | 4 | 40% | 5.6 | 3000 | 1140.68 |




A análise dos dados históricos compilados na Tabela 4 e ilustrados nos histogramas revela uma produtividade média de aproximadamente 776 LOC/Pessoa-Mês. Observa-se, contudo, uma variância natural dependente da tecnologia e características dos projetos passados: enquanto projetos intensivos em lógica de base de dados (como o Projeto 2) apresentaram valores superiores, aqueles com maior complexidade algorítmica ou de integração (como o Projeto 1 e 3) resultaram numa produção de código por unidade de esforço mais contida.

Considerando que o projeto atual envolve desafios de sistemas distribuídos e novas tecnologias de backend e frontend, a equipa optou por adotar uma postura conservadora para

mitigar riscos de subestimação. Assim, será considerada para os cálculos das secções seguintes uma Produtividade Representativa de 750 LOC/Pessoa-Mês.

## 2.3. Estimativas baseadas em linhas de código

Com base na divisão que foi feita para os casos de uso na etapa anterior dividimos o projeto nos seguintes componentes: Backend, Casos de uso do gestor, Casos de uso do motorista, Casos de uso do cliente e Frontend.

| Componente | Estimativa pessimista | Estimativa provável | Estimativa otimista |
|---|---|---|---|
| **Backend** | 3000 | 2000 | 1500 |
| **Gestor** (US 1,2,3,12,13,14,15,16) | 1700 | 1300 | 1100 |
| **Motorista** (US 4,5,7,8,10,11) | 1400 | 1100 | 1000 |
| **Cliente** (US 6,9) | 500 | 350 | 250 |
| **Frontend** | 700 | 450 | 350 |
| **Total** | 7300 | 5200 | 4200 |

Tendo em conta os dados históricos da secção 2.2 podemos concluir que o esforço médio do nosso grupo ronda os 776 LOC/Esforço.

Considerando a estimativa provável de 5200 linhas de código e a produtividade histórica da equipa de 776 LOC/Pessoa-Mês, estima-se um esforço linear de desenvolvimento de aproximadamente 6,7 Pessoas-Mês.

## 2.4. Estimativas baseadas no modelo COCOMO

Tendo por base o total de linhas de código obtido na secção anterior (Estimativa Provável), recorreu-se ao modelo **COCOMO II** (Constructive Cost Model) [2] para refinar a estimativa de esforço e tempo de desenvolvimento.

O cálculo foi efetuado considerando um tamanho de software de **5.2 KSLOC** (5200 linhas de código). Para a calibração do modelo, foram definidos os seguintes:

**Fatores de Escala**

Precedentedness: **Nominal** - A equipa compreende o domínio do problema, embora seja a primeira vez que implementa este sistema específico.

Development Flexibility: **High** - Tratando-se de um projeto académico, existe flexibilidade para ajustar requisitos não críticos face aos prazos.

Team Cohesion: **Very High** - A nossa equipa de 5 elementos trabalha de forma integrada e já demonstrou boa cooperação na Etapa 1.

**Geradores de Custo**

*Language and Tool Experience:* **High** – A equipa optou por Java (Spring Boot) e React, tecnologias com as quais possui familiaridade e experiência prévia elevada.

**Resultados do COCOMO II**

Com estes parâmetros, o modelo devolveu as seguintes estimativas:

**Esforço Estimado:** 11.0 Pessoas-Mês

**Duração Recomendada:** 7.7 Meses

**Análise Crítica**

Comparando os resultados do modelo COCOMO II com as estimativas anteriores, observam-se as seguintes conclusões:

O esforço calculado pelo COCOMO (11.0 PM) é superior ao estimado linearmente na Secção 2.3 (6.7 PM). Esta diferença justifica-se pelo facto de o COCOMO incluir nas suas contas o tempo de gestão, planeamento, *design* e testes, enquanto a estimativa linear baseada em LOC foca-se maioritariamente na codificação pura.

O facto de a duração sugerida (7.7 meses) exceder o calendário letivo (3 meses) e a disponibilidade horária ser limitada indica um risco elevado de não conclusão de todas as funcionalidades.

# 3. Processo de desenvolvimento de *software*

Na escolha do modelo de desenvolvimento de software que mais se adequa ao nosso projeto, tivemos em conta as características da nossa equipa e o sistema que iremos desenvolver. Para fundamentar a decisão, considerámos desde início alguns critérios importantes, tais como o facto de se tratar de uma aplicação web com requisitos que podem evoluir e de a equipa ser composta por cinco elementos com disponibilidade limitada. Além disso, o projeto será desenvolvido ao longo de dois semestres, o que exige uma abordagem flexível e que permita acompanhar progressos ao longo do tempo.

Assim sendo começamos por analisar as duas abordagens gerais existentes no desenvolvimento de software:

Os modelos prescritivos caracterizam-se por seguirem uma sequência bem definida de etapas, com planeamento completo antes da sua implementação. Têm pouca flexibilidade para alterações, mas facilitam a organização e a documentação do projeto.

Os modelos ágeis são baseados em ciclos iterativos curtos e adaptam-se facilmente a mudanças nos requisitos. Estes modelos valorizam a comunicação constante dentro da equipa e a colaboração com o cliente.

Após a análise das duas abordagens, concluímos que os modelos ágeis são mais adequados ao nosso projeto, uma vez que se trata de uma aplicação web com requisitos que podem sofrer ajustes ao longo do desenvolvimento. Além disso, a equipa é reduzida(5 elementos) e será necessário trabalhar por partes, o que favorece um modelo mais flexível.

Entre todas as opções de modelos ágeis que existem, decidimos optar por utilizar o modelo de scrum, que é muito utilizado na indústria de software devido à sua simplicidade, eficácia e capacidade de adaptação.Este modelo organiza o trabalho em sprints, que são interações curtas de desenvolvimento que incluem planeamento, implementação, testes e revisão.Cada sprint resulta numa entrega funcional, o que permite demonstrar progressos de forma regular e corrigir possíveis problemas de forma antecipada. O número de sprints depende da complexidade e tamanho do sistema.Como foi referido no Scrum Guide, *"Sprints are the heartbeat of Scrum, where ideas are turned into value."*.

| Vantagens | Desvantagens |
|---|---|
| Permite organizar o trabalho em partes menores | Requer disciplina e compromisso por parte de todos os elementos |
| Promove uma gestão ativa e colaborativa da equipa por meio de reuniões regulares | Exige reuniões frequentes, o que pode ser difícil devido às diferentes disponibilidades da equipa |
| Permite ter resultados regulares ao longo do projeto | A má definição do backlog pode comprometer os sprints seguintes |
| Ajuda a estabelecer prioridades claras para cada sprint | A dependência entre tarefas pode comprometer o progresso caso algum membro falhe |

# 4. Organização da equipa

Tendo em consideração que a equipa é constituída por 5 elementos e que o processo de desenvolvimento adotado é o Scrum (descrito no Capítulo 3), o paradigma de organização escolhido é democrático e descentralizado, com uma estrutura horizontal. Esta escolha justifica-se pela necessidade de agilidade e comunicação constante neste que é um projeto de curta duração, permitindo que, embora existam especializações, todos os membros mantenham uma visão integrada do projeto.

Para agilizar o desenvolvimento e permitir que as tarefas avancem em simultâneo, organizámos a equipa em três grupos: um dedicado ao Frontend e dois ao Backend. A atribuição destes papeis teve em conta as competências e as preferências de cada membro, conforme detalhado na Secção 1.1.

### Desenvolvimento Frontend

Dividimos o desenvolvimento do frontend por um elemento, garantindo a consistência visual e a experiência do utilizador.

#### Programador de Frontend e Responsável de Testes com Utilizadores: Martim Melro

É o responsável exclusivo pela implementação da interface do utilizador (*Client-side*) em React.js. As suas tarefas incluem o desenvolvimento de todos os componentes visuais, a integração com a API REST, a gestão do estado da aplicação e a validação de formulários no cliente. Acumula a função de coordenar os testes de usabilidade, garantindo que a aplicação cumpre os requisitos de interatividade e navegabilidade.

### Desenvolvimento Backend

Dividimos o desenvolvimento do backend em Java (Spring Boot) e a infraestrutura em duas equipas de dois elementos. Organizámos o trabalho por funcionalidades, o que ajuda a equilibrar o esforço de todos e a evitar conflitos no código.

**Equipa 1 (Gonçalo e Miguel): Infraestrutura, Segurança e Gestão de Entidades:** Esta equipa foca-se na fundação do sistema, garantindo a segurança, a persistência de dados e a gestão das entidades base (Utilizadores, Táxis, Motoristas).

**Administrador de Base de Dados e Gestor de Projeto:** Gonçalo Moutinho

Enquanto Administrador de Base de Dados, é responsável pelo desenho e manutenção do esquema da base de dados MySQL, garantindo a integridade referencial, a otimização de *queries* e a coerência dos dados das entidades principais (Táxis e Motoristas). Como Gestor de Projeto, facilita o cumprimento da metodologia Scrum e a remoção de impedimentos. Ao nível de desenvolvimento, implementa os serviços de gestão dos táxis e turnos.

**Especialista de Redes e Segurança:** Miguel Lemos

Foca-se nos requisitos não funcionais de infraestrutura. É responsável pela configuração dos serviços Cloud (AWS), implementação da arquitetura de rede, configuração do **HAProxy** para balanceamento de carga e gestão das políticas de segurança (*Firewalls* e Grupos de Segurança). No desenvolvimento Backend, encarrega-se do módulo de Autenticação e Autorização (Spring Security), garantindo o acesso seguro à API.

**Equipa 2 (Francisco e Rodrigo): Lógica de Viagens, Faturação e Lógica do Negócio:** Esta equipa foca-se na lógica de negócio, implementando o ciclo de vida das viagens, o processamento do pagamento e das faturas e as estatísticas.

**Arquiteto de Sistema e Programador Backend:** Francisco Parasca

Responsável pela implementação da base da aplicação: o algoritmo de pedidos de táxi e gestão de viagens (US 6 a 8). Isto inclui a lógica de contacto entre cliente e motorista, o cálculo de estados da viagem e a orquestração dos serviços de backend. Como arquiteto, garante que os módulos desenvolvidos pelas duas equipas de backend comunicam de forma eficiente e seguem os padrões REST definidos.

**Consultor de Dados, Técnico de Manutenção e Programador Backend:** Rodrigo Sequeira

Responsável pelo desenvolvimento da Faturação (US 9 e 10) e Relatórios (US 14 a 16), implementando a lógica de cálculo de preços e agregação de dados estatísticos. Enquanto consultor de dados assegura a validação das regras de negócio complexas e supervisiona a qualidade do código e a documentação técnica (*Javadoc*) do projeto.

# 5. Planeamento do projeto

Antes de termos iniciado a elaboração dos Work Packages,voltamos a fazer o planeamento definido na primeira fase, garantindo a sua coerência com objetivo final do projeto e com a organização da equipa. O ficheiro encontra-se incluído na entrega, com o nome "*planeamentopgp_correcao1fase*.pdf".

## 5.1. *Work packages*

Os Work Packages (WPs) seguem o processo Scrum descrito no Capítulo 3. Cada Work Package agrupa um conjunto de tarefas relacionadas permitindo a divisão do projeto em diferentes blocos, facilitando assim o trabalho de equipa e reduzindo as dependências entre as partes do sistema.

Cada WP contém as tarefas, recursos, dependências, intervalo de tempo e resultados esperados garantindo um planejamento simples e estruturado.A distribuição dos recursos pelos Work Packages segue a estrutura organizacional definida anteriormente no capítulo 4.

### WP1 – Infraestrutura Inicial e API Base

**Descrição:** Preparação do ambiente técnico, criação da base de dados, configuração de autenticação e desenvolvimento da API base necessária para suportar todos os módulos do sistema.

**Recursos:**

- **Gonçalo Moutinho (DBA & Gestor de Projeto)** – Modelação da base de dados e coordenação do WP.
- **Miguel Lemos (Infraestrutura & Segurança)** – Configuração da cloud, rede, HAProxy e componentes iniciais da arquitetura.
- **Francisco Parasca (Backend & Arquitetura)** – Estrutura da API base e endpoints fundamentais.

**Justificação:** Este WP exige conhecimentos combinados de infraestrutura, bases de dados e backend.

**Intervalo temporal:** 11 dias (02/02 – 16/02)
**Dependências:** Nenhuma .

**Tarefas:**

1. Configuração inicial da infraestrutura
2. Modelação e criação da Base de Dados
3. Configuração inicial da autenticação
4. Desenvolvimento da API base
5. Plataforma preparada para funcionalidades futuras

**Resultados esperados:**

- Base de dados implementada
- API base funcional
- Autenticação inicial
- Infraestrutura pronta para receber os módulos seguintes

### WP2 – Funcionalidades do Gestor

**Descrição:** Implementação de todas as funcionalidades administrativas: registo de táxis/motoristas, tarifação, gestão de turnos e interface do gestor.

**Recursos:**

- **Francisco Parasca (Backend & Arquitetura)** – Implementação das regras administrativas e endpoints REST.
- **Martim Melro (Frontend)** – Interface administrativa e fluxos de gestão.

- **Rodrigo Sequeira e Gonçalo Moutinho (Dados & Backend)** – Validação de dados e consistência das regras administrativas.

**Justificação:** Este WP reúne o CRUD administrativo e gestão de entidades, que combina backend estruturado, validação de dados e uma interface visual consistente.

**Intervalo temporal:** 17 dias (17/02 – 11/03)
**Dependências:** WP1

**Tarefas:**

1. Registo de táxis e motoristas (US1, US2)
2. Definição de preços e políticas de tarifação (US3)
3. Gestão de turnos pela administração (US5)
4. Edição e remoção de dados (US12, US13)
5. Interface administrativa do Gestor

**Resultados:**

- Módulo administrativo totalmente operacional
- Todas as entidades e regras administrativas implementadas
- Interface funcional para o Gestor

## WP3 – Funcionalidades do Motorista

**Descrição:** Desenvolvimento do módulo operacional usado pelos motoristas: autenticação, gestão de turnos, viagens, telemetria e reabastecimentos.

**Recursos:**

- **Francisco Parasca (Backend)** – Implementação da lógica de turnos, viagens e telemetria.
- **Martim Melro (Frontend)** – Interface do motorista.
- **Gonçalo Moutinho (Dados)** – Verificação da integridade e coerência das operações do motorista.

**Justificação:** Toda a lógica de viagens (Equipa 2) e o UI do motorista são competências diretas destes elementos.

**Intervalo temporal:** 19 dias (12/03 – 7/04)
**Dependências:** WP2

**Tarefas:**

1. Autenticação + painel inicial (US4)
2. Gestão de início/fim de turno (US5)
3. Processo de viagem + telemetria (US8)
4. Cálculo automático de custo (US8)
5. Reabastecimentos (US11)

**Resultados esperados:**

- Aplicação funcional para motoristas

- Registo completo de viagens, custos e turnos

### WP4 – Frontend e Interação com o Cliente

**Descrição:** Desenvolvimento do fluxo de cliente: pedido de táxi, aceitação, rotas, pagamentos, faturas e testes de usabilidade.

**Recursos:**

- **Martim Melro (Frontend/UX)** – Página de pedido, rotas, estado da viagem e interação do utilizador.
- **Francisco Parasca (Backend)** – Lógica de pedidos, emparelhamento cliente ↔ motorista e rotas.
- **Miguel Lemos (Infraestrutura/APIs externas)** – Integração com sistemas de geolocalização.

**Justificação:** Este WP exige ligação entre UX, lógica de viagem e serviços externos.

**Intervalo temporal:** 19 dias (8/04 – 4/05)
**Dependências:** WP3

**Tarefas:**

1. Pedido de táxi (US6)
2. Aceitação de pedido (US7)
3. Geolocalização e rotas (US6, US7)
4. Pagamento digital (US9)
5. Emissão de fatura (US10)
6. Testes de usabilidade

**Resultados esperados:**

- Fluxo completo de cliente validado
- Pagamento e emissão de faturas automáticos

### WP5 – Relatórios e Consolidação Administrativa

**Descrição:** Desenvolvimento da camada analítica do sistema: relatórios administrativos, financeiros e operacionais.

**Recursos:**

- **Francisco Parasca (Backend)** – Geração das consultas e endpoints dos relatórios.
- **Rodrigo Sequeira e Gonçalo Moutinho (Consultor de Dados)** – Verificação de cálculos, agregações e consistência estatística.
- **Martim Melro (Frontend)** – Implementação do painel de relatórios.

**Justificação:** Os relatórios envolvem lógica de dados, backend para processamento e apresentação visual .

**Intervalo temporal:** 12 dias (5/05 – 20/05)
**Dependências:** WP4

**Tarefas:**

1. Relatório de atividade de táxis (US14)
2. Relatório financeiro (US15)
3. Relatório de custos e reabastecimentos (US16)
4. Melhorias finais ao painel administrativo

**Resultados esperados:**

- Painel completo de relatórios
- Consolidação analítica do sistema

## WP6 – Segurança e Integrações Externas

**Descrição:** Implementação dos mecanismos de segurança e comunicações seguras, bem como integração externa (APIs de mapas e geolocalização).

**Recursos:**

- **Miguel Lemos (Segurança & Redes)** – TLS, firewall, balanceamento e políticas de proteção.
- **Francisco Parasca (Backend)** – Autenticação avançada e hardening da API.
- **Rodrigo Sequeira (Qualidade)** – Testes e validação dos mecanismos de segurança.

**Justificação:** O WP combina infraestrutura segura e validação lógica

**Intervalo temporal:** 8 dias (21/05 – 1/06)
**Dependências:** WP5

**Tarefas:**

1. Autenticação avançada
2. Configuração de TLS/HTTPS
3. Firewall e proteção de endpoints
4. Integração de APIs externas

**Resultados esperados:**

- Sistema em conformidade com requisitos de segurança
- API exposta de forma segura
- Rotas e geolocalização totalmente funcionais

## WP7 – Testes, Automação e Entrega Final

**Descrição:** Automação CI/CD, testes (unitários, carga, regressão), revisão final e apresentação do projeto.

**Recursos:**

- **Miguel Lemos (DevOps)** – Pipelines CI/CD e deployment automático.
- **Rodrigo Sequeira (Qualidade & Dados)** – Testes unitários, integração e carga.
- **Martim Melro & Francisco Parasca** – Correção final e melhorias no frontend/backend.
- **Gonçalo Moutinho (Gestor de Projeto)** – Coordenação da entrega e apresentação final.

**Justificação:** Este WP exige competências de DevOps, QA e coordenação final.

**Intervalo temporal:** 11 dias (2/06 – 16/07)
**Dependências:** WP6

**Tarefas:**

1. Configuração de pipelines CI/CD
2. Testes unitários, integração e regressão
3. Testes de carga
4. Validações finais
5. Entrega e apresentação

**Resultados esperados**

- Sistema estável, testado e automatizado
- Release final pronta para entrega

## 5.2. Calendarização

O mapa de Gantt e a tabela de horas de trabalho não se encontram incorporados neste relatório, uma vez que a sua exportação em formato PDF resultava na desformatação significativa do conteúdo. Para garantir uma visualização adequada, o mapa está disponibilizado como ficheiro separado, com o nome “mapa_Gantt.pdf” e a tabela no ficheiro, com o nome ”TabelaHoras.pdf”.

## Milestones do Projeto

- **M1 – Infraestrutura e API base concluídas (16/02/2026)**
  Entregue:O cliente recebe a plataforma técnica funcional.

- **M2 – Módulo administrativo concluído (11/03/2026)**
  Entregue:Gestão de táxis, motoristas e tarifação operacional.

- **M3 – Módulo do motorista funcional (07/04/2026)**
  Entregue:Turnos, viagens, custos e telemetria implementados.

- **M4 – Módulo do cliente concluído (04/05/2026)**
  Entregue:Clientes podem pedir táxi, pagar e receber fatura.

- **M5 – Relatórios finalizados (20/05/2026)**
  Entregue:O administrador pode monitorizar a atividade e as finanças.

- **M6 – Segurança implementada (01/06/2026)**
  Entregue:API protegida com TLS, firewall e autenticação avançada.

- **M7 – Entrega final do sistema (16/06/2026)**
  Entregue:Versão testada, estável e finalizada.

## 6. Gestão de riscos

A gestão de riscos exige a identificação de todos os possíveis fatores que possam influenciar negativamente o sucesso do projeto.

O objetivo desta seção é apresentar,identificar ,comentar e definir maneiras de prevenir os riscos, estimando a sua probabilidade e impacto iniciais. o objetivo não é esconder e fugir dos riscos, mas sim reduzir a sua probabilidade de aparecerem e influenciarem a execução do projeto.

### 6.1. Lista de riscos

Os riscos foram classificados em 3 categorias principais, que correspondem aos tipos de risco mais comuns em projetos de software:

- -Risco de Projeto (PR): Afetam o planeamento, o orçamento, o calendário e os recursos humanos
- -Risco Técnico (TE): Afetam a qualidade e implementação do produto
- -Risco de Negócio (NE): Afetam a viabilidade do produto no mercado

Adicionalmente, os riscos foram classificados quanto à sua dimensão como:

- -Geral: Inerentes a qualquer projeto de software
- -Específico: Ligado às tecnologias e arquitetura deste projeto

Riscos Gerais

| Risco (Categoria) | Descrição | Consequência Principal |
|---|---|---|
| **Rotatividade de Pessoal na Equipa (PR)** | Possibilidade de perda de membros da equipa durante o ciclo de vida do projeto. | Atraso na calendarização e necessidade de realocação de esforço para formação e execução das tarefas pendentes. Agravado pela agenda intensa de alguns membros. |
| **Definição de Responsabilidades Vagas (PR)** | Definição ou atribuição incompleta de papéis e responsabilidades na equipa. | Tarefas não realizadas ou excesso de esforço desnecessário na sua execução. |
| **Integração de Software Reutilizado de Baixa Qualidade (TE)** | Código *open-source* ou bibliotecas de terceiros inseguras, de fraca qualidade ou | Falha de funcionamento, incumprimento dos requisitos de qualidade, confiabilidade e |

| | | |
|---|---|---|
| | desatualizadas. | manutenção. |
| **Não Cumprimento dos Requisitos de Performance (TE)** | O sistema pode não satisfazer os requisitos de desempenho, como a baixa latência (RNF-07), sob carga. | Embora mitigado pela arquitetura de replicação (x2) e *Load Balancers* (HAProxy), a lógica de negócio ineficiente pode levar a falha de missão. |
| **Problemas na Integração Frontend/Backend (React.js/Springboot ) (TE)** | Dificuldade em estabelecer e manter uma comunicação eficiente e estável entre o Frontend (React.js) e a API Rest (**Springboot** ). | Atrasos e *bugs* persistentes na aplicação. |

Riscos Específicos

| Risco (Categoria) | Descrição | Consequência Principal |
|---|---|---|
| **Estimativa de Esforço Insuficiente (PR)** | O esforço total estimado para desenvolver a aplicação pode ser insuficiente. | Atrasos no projeto devido à incerteza estrutural (novas tecnologias, integração de múltiplas APIs) que dificulta a estimativa de tempo e linhas de código (LOC). |
| **Limitações das APIs de Geolocalização/Rotas Gratuitas (TE)** | Utilização de serviços *open-source* (Nominatim e OSRM) que podem apresentar falhas de escala ou restrições de uso sob tráfego elevado. | Afeta criticamente funcionalidades essenciais como cálculo de rotas, distância e preço final da viagem. |
| **Curva de Aprendizagem do Springboot e React.js (TE)** | Inexperiência da equipa com a *framework* **Springboot** (Backend) e a biblioteca React.js (Frontend). | Atraso nos *Work Packages* e introdução de erros de desenho de sistema, especialmente dada a curva de aprendizagem inicial elevada do **Springboot** . |
| **Limitações de Escala com o SGDB Postgress(TE)** | Embora familiar para a equipa, o **Postgress** pode ter dificuldades em lidar com grandes cargas. | Problemas de desempenho e latência na geração de relatórios (US 14, 15, 16). |

| | | |
|---|---|---|
| **Falha na Integração de Pagamentos Online (Stripe) (NE)** | Dependência total da API de Pagamentos Online (Stripe). | Qualquer indisponibilidade ou falha na integração pode impedir a conclusão de transações (US 9), inviabilizando o modelo de negócio. |
| **Falha de Segurança/Violação de Dados (Postgress) (NE)** | Ciberataque ou falha de segurança que resulte na violação de dados sensíveis (NIF, moradas), apesar das medidas de segurança implementadas. | Penalidades legais (RGPD) e perda de confiança. |

Nota: (PR) - Risco de Projeto, (TE) - Risco Técnico, (NE) - Risco de Negócio/Legal.

## 6.2. Tabela de riscos

A Tabela de Riscos tem como objetivo classificar os riscos identificados por ordem decrescente de prioridade, determinada pela conjugação da sua Probabilidade (em percentagem) e do seu Impacto (em escala de 1 a 4, onde 1 significa "Catastrófico" e 4 "Negligível"). Esta análise é crucial para orientar o Plano RMMM (Risk Mitigation, Monitoring, and Management) para as áreas de maior vulnerabilidade do projeto.

Este grupo inclui os riscos com maior probabilidade de ocorrência ou com impacto mais severo, exigindo ações de Mitigação imediatas.

| Nº | Risco (Categoria) | Probabilidade | Impacto (Numérico / Descritivo) | Justificação |
|---|---|---|---|---|
| **1** | **Rotatividade de Pessoal na Equipa (PR)** | 80% | 2 (Crítico) | Elevada possibilidade de perda de membros, intensificada por atividades paralelas, conferindo a mais alta prioridade devido à alta probabilidade e ao impacto Crítico na calendarização. |
| **2** | **Limitações das APIs de Geolocalizaçã o/Rotas Gratuitas (TE)** | 60% | 2 (Crítico) | A dependência de serviços *open-source* (Nominatim e OSRM) sem garantia de escala torna altamente provável a falha do sistema em tráfego |

| | | | | |
|---|---|---|---|---|
| | | | | elevado, afetando criticamente funcionalidades essenciais (ex: cálculo do preço final da viagem). |
| **3** | **Estimativa de Esforço Insuficiente (PR)** | 70% | 2 (Crítico) | A incerteza estrutural devido a novas tecnologias e à integração de múltiplas APIs confere alta probabilidade de falha na estimativa, resultando em atrasos no projeto. |
| **4** | **Curva de Aprendizagem do Springboot e React.js (TE)** | 60% | 3 (Marginal) | A inexperiência da equipa com a *framework* **Springboot** torna provável (60%) que ocorram atrasos nos *Work Packages* (WPs) e erros de desenho do sistema, com impacto Marginal. |

Estes riscos apresentam menor probabilidade, mas o seu Impacto pode ser **Crítico** (2) ou **Marginal** (3) para a qualidade ou viabilidade do produto.

| **Nº** | **Risco (Categoria)** | **Probabilidade** | **Impacto (Numérico / Descritivo)** | **Justificação** |
|---|---|---|---|---|
| **5** | **Não Cumprimento dos Requisitos de Performance (TE)** | 50% | 2 (Crítico) | Probabilidade Média de a lógica de negócio ser ineficiente sob carga. O impacto é Crítico, pois a falha de desempenho pode levar a falha de missão. |

| | | | | |
|---|---|---|---|---|
| **6** | **Falha na Integração de Pagamentos Online (Stripe) (NE)** | 50% | 2 (Crítico) | A dependência total da API do **Stripe** confere uma probabilidade de falha Média, mas a interrupção das transações inviabiliza o modelo de negócio (Impacto Crítico). |
| **7** | **Falha de Segurança/Violação de Dados (Postgress) (NE)** | 50% | 2 (Crítico) | O risco de ciberataque ou falha de segurança com violação de dados sensíveis tem Impacto Crítico (incumprimento do RGPD e perda de confiança). |
| **8** | **Problemas na Integração Frontend/Backend (React.js/Springboot ) (TE)** | 45% | 3 (Marginal) | Probabilidade Média de dificuldade em estabelecer comunicação eficiente entre o **React.js** e a API Rest do **Springboot**, resultando em atrasos e *bugs* persistentes. |
| **9** | **Definição de Responsabilidades Vagas (PR)** | 40% | 3 (Marginal) | Falha na gestão (probabilidade Média) resultaria em ineficiência e excesso de esforço desnecessário (Impacto Marginal). |
| **1** | **Limitações de Escala com o SGDB Postgress(TE)** | 40% | 3 (Marginal) | A dificuldade do **Postgress** em lidar com grandes cargas pode levar a problemas de |

| | | | | |
|---|---|---|---|---|
| | | | | desempenho e latência na geração de relatórios (US 14, 15, 16). |
| 1 | **Integração de Software Reutilizado de Baixa Qualidade (TE)** | 40% | 3 (Marginal) | Probabilidade Média de código *open-source* apresentar falhas ou incumprir requisitos de qualidade, com impacto marginal no funcionamento do produto. |

-----**Linha de Corte e Foco na Mitigação (Plano RMMM)**

De acordo com as diretrizes do projeto, a **Linha de Corte** é estabelecida após o **5º risco** de maior prioridade. Estes riscos concentram as ameaças mais críticas para o sucesso do projeto e serão o foco principal do **Plano RMMM** (Secção 6.3).

**Os 5 Riscos Acima da Linha de Corte são:**

- Rotatividade de Pessoal na Equipa (PR)
- Limitações das APIs de Geolocalização/Rotas Gratuitas (TE)
- Estimativa de Esforço Insuficiente (PR)
- Curva de Aprendizagem do **Springboot** e React.js (TE)
- Não Cumprimento dos Requisitos de Performance (TE)

## 6.3. Plano RMMM

Com base na Tabela de Riscos (Secção 6.2), esta secção detalha o Plano RMMM, concentrando-se nos **5 riscos de maior prioridade**. O plano estabelece ações específicas de prevenção (Mitigação), deteção precoce (Monitorização) e resposta controlada (Gestão/Contingência) para garantir a exequibilidade do projeto.

Rotatividade de Pessoal na Equipa (PR)

| **Categoria** | **Descrição** |
|---|---|

| | |
|---|---|
| **Mitigação** | **Equipa de Dupla Alocação:** Estruturar a equipa em pares (2x2) para garantir que as atividades críticas e os módulos de código sejam do conhecimento de, pelo menos, dois elementos. **Documentação de Processo:** Implementar a obrigatoriedade de criação de Javadoc e a manutenção rigorosa de documentação técnica para o caso de falha de um membro individual. |
| **Monitorização** | Avaliar a qualidade e a frequência das *code reviews* entre os pares para confirmar a eficácia da partilha de conhecimento. Acompanhar a disponibilidade semanal efetiva de cada membro face ao plano. |
| **Gestão** | **Substituição Direta:** Se um elemento de um par falhar, o colega assume o trabalho. Se o falhar for um membro singular, um par é desmembrado temporariamente, e um membro é designado para assumir as tarefas, sem necessidade de *setup* ou formação prévia. **Reajuste:** O Gestor deve reajustar o calendário, alocando tempo adicional para que o par absorva a nova carga de trabalho. |

Estimativa de Esforço Insuficiente (PR)

| **Categoria** | **Descrição** |
|---|---|
| **Mitigação** | **Validação COCOMO II:** Recorrer ao modelo **COCOMO II** para validar a estimativa inicial do esforço. **Scrum:** Utilizar o *framework* **Scrum** para reavaliar as prioridades no final de cada *Sprint*. |
| **Monitorização** | Comparar o **esforço real com o esforço estimado em cada sprint**. |
| **Gestão** | **Replaneamento:** Efetuar um replaneamento da calendarização dos *sprints* seguintes. |

Limitações das APIs de Geolocalização/Rotas Gratuitas (TE)

| Categoria | Descrição |
| --- | --- |
| **Mitigação** | **Fallback de Distância:** Implementar a **Fórmula Haversine** como *fallback* imediato para o cálculo da distância em linha reta. |
| **Monitorização** | Monitorizar a **taxa de falha** e a **latência** das chamadas às APIs externas. Controlar o volume de pedidos para não exceder os limites de uso das APIs gratuitas. |
| **Gestão** | **Aproximação Temporal:** Em caso de falha ou limite atingido, utilizar a aproximação de 4 minutos por km como último recurso. |

Curva de Aprendizagem do Springboot e React.js (TE)

| Categoria | Descrição |
| --- | --- |
| **Mitigação** | Dedicar o **primeiro sprint** à criação de uma prova de conceito para uma integração estável do *frontend* e *backend*, incluindo a persistência da base de dados, validando o *setup* antes de iniciar a lógica de negócio. |
| **Monitorização** | Acompanhar o número de *bugs* de arquitetura e o tempo despendido na implementação da camada de persistência. |
| **Gestão** | **Revisão Tecnológica:** Em caso de atraso significativo, considerar a simplificação da arquitetura **API RESTful**, reduzindo a complexidade de implementação. |

Não Cumprimento dos Requisitos de Performance (TE)

| Categoria | Descrição |
| --- | --- |
| **Mitigação** | **Otimização do SGDB:** Otimizar *queries* e índices no **PostgreSQL**. |
| **Monitorização** | **Testes de Carga:** Realizar testes de carga para medir as **latências das API** e garantir o cumprimento do **RNF-07**. **Monitorização de Recursos:** Acompanhar o uso de CPU |

| | |
|---|---|
| | e memória nos servidores. |
| **Gestão** | **Otimização do Código:** Concentrar os esforços na **otimização rigorosa do código** (algoritmos mais eficientes e redução de *queries* à base de dados), em vez de recorrer à escalabilidade de infraestrutura, para manter a *performance* dentro dos limites estabelecidos. |

# Conclusão

O presente relatório detalhou o planeamento para a implementação da aplicação de gestão de táxis, cobrindo desde a alocação de recursos até à gestão de risco.

Da análise das estimativas efetuadas, conclui-se que o projeto é **viável**, mas apresenta desafios significativos em termos de prazo. A comparação entre o esforço disponível pela equipa (Secção 2.1) e o esforço estimado pelo método de decomposição (Secção 2.3) e pelo modelo COCOMO (Secção 2.4) revela uma margem de manobra reduzida. A equipa possui as competências técnicas necessárias, suportadas pela escolha de tecnologias robustas como React e springboot, mas a carga de trabalho exigirá disciplina.

A principal ameaça à exequibilidade reside na sobreposição de prazos com outras unidades curriculares, identificada como um risco de projeto prioritário no Capítulo 6.

Para mitigar este facto, a adoção da metodologia **Scrum** será fundamental, permitindo entregas incrementais e a reavaliação constante das prioridades. Caso se verifiquem atrasos irrecuperáveis, a estratégia de contingência passará pela renegociação do âmbito (redução das *User Stories* opcionais ou de menor prioridade), garantindo que os requisitos obrigatórios e o fluxo principal (registar motorista, pedir táxi, efetuar viagem e faturar) estejam totalmente funcionais na data de entrega final.

# Bibliografia

[1] IntelliJ IDEA Documentation. Acesso em Dezembro 2025.
https://www.jetbrains.com/help/idea/getting-started.html

[2] Oracle. Java Documentation. Acesso em Dezembro 2025.
https://docs.oracle.com/en/java/javase/index.html

[3] ISO/IEC 9075 (SQL Standard). Acesso em Dezembro 2025.
https://www.iso.org/standard/63555.html

[4] MDN Web Docs. JavaScript. Acesso em Dezembro 2025.
https://developer.mozilla.org/pt-BR/docs/Web/JavaScript

[5] React Documentation. Acesso em Dezembro 2025. https://react.dev/documentation

[6] Leaflet Documentation. Acesso em Dezembro 2025. https://leafletjs.com/reference.html

[7] Spring Boot Reference Documentation. Acesso em Dezembro 2025.
https://docs.spring.io/spring-boot/docs/current/reference/html/

[8] Spring Data Documentation. Acesso em Dezembro 2025.
https://docs.spring.io/spring-data/commons/docs/current/reference/html/

[9] Spring Security Reference. Acesso em Dezembro 2025.
https://docs.spring.io/spring-security/reference/index.html

[10] Visual Studio Code Documentation. Acesso em Dezembro 2025.
https://code.visualstudio.com/docs

[11] Stripe API Reference. Acesso em Dezembro 2025. https://stripe.com/docs/api

[12] Firebase Authentication Documentation. Acesso em Dezembro 2025.
https://firebase.google.com/docs/auth

[13] Nominatim Documentation. Acesso em Dezembro 2025.
https://nominatim.org/release-docs/latest/

[14] OSRM API Documentation. Acesso em Dezembro 2025.
http://project-osrm.org/docs/v5.22.0/api/

[15] JUnit 5 User Guide. Acesso em Dezembro 2025.
https://junit.org/junit5/docs/current/user-guide/

[16] Javadoc Tool Documentation. Acesso em Dezembro 2025.
https://docs.oracle.com/en/java/javase/21/docs/specs/javadoc/doc-comment-spec.html

[17] OpenAPI Specification. Acesso em Dezembro 2025. https://swagger.io/specification/

[18] Git Documentation. Acesso em Dezembro 2025. https://git-scm.com/docs

[19] Discord Support. Acesso em Dezembro 2025. https://support.discord.com/hc/pt-br